

# Práctica P1

## Navegación en entornos complejos

Mapeo, navegación y teleoperación de una aspiradora

Robótica de Servicios · Curso 2026/2027

Alejandro Lavandeira Casais  
Carlos Adrián Cancela Bodlak  
Yago Martínez Pena

### Resumen

Esta práctica consiste en conseguir que una aspiradora explore un piso, se desplace entre dos puntos y repita una ruta marcada por el usuario. El robot tiene un cuerpo de 2x2 unidades y avanza 0,5 unidades por ciclo. Para decidir si puede moverse utiliza sus sensores de contacto y su posición simulada.

En A guarda las posiciones que descubre y crea la matriz cuando termina de explorar. En B busca un camino en ese mapa con A\*. En C permite conducir con el teclado, guardar puntos de paso y volver a recorrerlos. Los giros manuales son de 90 grados y los puntos se muestran con una X numerada.

Las pruebas de los cuatro escenarios originales no dejan celdas alcanzables sin cubrir. Las rutas de navegación y la reproducción ensayadas terminan sin colisiones registradas. Estos resultados tienen límites: no se descubren habitaciones desconectadas y el simulador no introduce errores de posición.

Entorno de ejecución: Python 3.12 y Pygame 2.6.1.

# 1. Diseño de la práctica

El enunciado divide el trabajo en tres partes. A debe descubrir y recorrer el entorno; B debe ir de un origen a un destino sorteando obstáculos; C debe guardar y recuperar rutas construidas mediante teleoperación. Se utiliza la opción de planificación sobre un mapa para B y la de marcar puntos de paso para C.

## Qué información conoce el robot

La odometría es la estimación de posición y orientación que proporcionan los sensores de movimiento. Aquí coincide con la posición real del simulador: no hay ruido ni errores acumulados. Los sensores de contacto indican si el siguiente avance está bloqueado, pero no dan la distancia a un mueble ni su forma completa.

El simulador conoce el plano para dibujarlo y producir los sensores. El explorador no recibe su ancho, alto ni lista de muebles. Guarda coordenadas mientras recorre el piso; al terminar calcula los límites de lo observado y reserva la matriz.

## Organización del código

La tabla 1 indica dónde está cada tarea. Los tres scripts llaman a funciones compartidas: así B y C ejecutan los caminos con el mismo controlador, en lugar de tener dos formas distintas de mover el robot.

| Parte     | Qué hace                                            | Dónde se implementa                                             |
|-----------|-----------------------------------------------------|-----------------------------------------------------------------|
| Simulador | Carga el mundo, genera sensores y mueve el robot.   | rds2026environment.py, rds2026machines.py, rds2026simulation.py |
| A         | Explora, guarda observaciones y crea el mapa final. | apartado_a.py; SparseExplorationMap y CompleteCoverageExplorer  |
| B         | Valida extremos, busca el camino y lo ejecuta.      | apartado_b.py; astar y MotionController                         |
| C         | Graba puntos y planifica entre ellos al reproducir. | apartado_c.py; RecordedRoute y WaypointOverlay                  |
| Pruebas   | Comprueba funciones y mide recorridos completos.    | tests/test_practica.py y validar_practica.py                    |

Tabla 1. Distribución de responsabilidades. Las clases y funciones compartidas están en `robotica_servicios.py`.

A partir del simulador original se corrigió el estado compartido entre instancias y el redondeo de la posición futura. Este último podía permitir un paso al ir y bloquearlo al volver. Son cambios del modelo simulado; no aportan al explorador información que todavía no haya observado.

## 2. Apartado A: descubrir y cubrir

### Posición del robot y celda de suelo

No son lo mismo. Una pose es una posición del centro del robot. Que sea libre significa que cabe el cuerpo completo. Una celda puede quedar cubierta por un lateral aunque el centro nunca pase por ella. La cobertura usa las mismas cuatro muestras bajo el cuerpo que el simulador; no calcula un área continua exacta.

### Antes de crear la matriz

SparseExplorationMap mantiene tres conjuntos sin duplicados: `fine_free` para centros visitados, `fine_obstacles` para centros bloqueados y `covered` para suelo cubierto. Las poses usan claves enteras: (21,5; 21) se representa como (43; 42), porque cada unidad de clave equivale a 0,5 unidades del mundo.

Al acabar, finalice calcula los mínimos y máximos de las celdas cubiertas y las poses observadas. El ancho es  $\max\_x - \min\_x + 1$ ; el alto se calcula igual. El origen guardado permite traducir coordenadas del mundo a filas y columnas. El TXT tiene 1 para suelo cubierto y 0 para suelo sin cobertura registrada. El JSON guarda también desconocido, libre y bloqueado (0, 1 y 2) y las poses de medio paso.

### Cómo recorre el entorno

La búsqueda en profundidad, o DFS, prueba los cuatro vecinos de cada pose. Si puede avanzar, guarda la nueva pose y continúa desde ella. Al agotar sus vecinos vuelve físicamente a la posición anterior. Una pila recuerda esas ramas pendientes; cuando queda vacía, termina. Esto recorre bordes e interior, pero no es un seguidor de paredes ni un barrido por filas.

Probar vecinos cada 0,5 unidades permite explorar desde posiciones intermedias que antes se atravesaban sin examinar sus salidas. La figura 1 muestra el suelo cubierto y las poses bloqueadas. El rojo marca centros donde no cabe el robot, no la silueta exacta de los muebles.

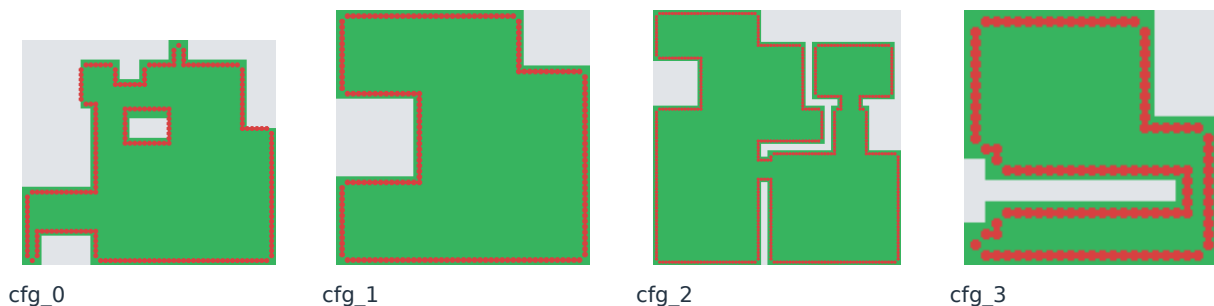


Figura 1. Mapas observados tras explorar. Verde: suelo con cobertura registrada. Rojo: poses bloqueadas del centro. Gris: suelo sin cobertura registrada; no demuestra que sea un obstáculo.

### 3. Apartado B: ir de un punto a otro

#### Comprobar los extremos

Antes de mover el robot se comprueban el origen y el destino. Un punto fuera del mapa, bloqueado o desconocido se rechaza. Si ambos son libres pero están separados por paredes sin paso, A\* informa de que no hay camino. No se sustituye el destino por otro cercano para dar la ruta por completada.

Los puntos entre muestras admiten un ajuste local solo si todos los vértices que los rodean son libres conocidos. Por ejemplo, una coordenada 2,2 puede aproximarse a 2 o 2,5. Se informa de la distancia del ajuste. Esto no demuestra llegada exacta a cualquier coordenada continua y también se aplica al origen.

#### Cómo decide A\*

Cada pose libre es un nodo y sus cuatro vecinos son las posibles transiciones. A\* prioriza la suma de los pasos ya recorridos y una estimación de los que faltan. Esa estimación suma las diferencias horizontal y vertical hasta el destino: es la distancia Manhattan. Al ignorar obstáculos no sobreestima el recorrido. A\* obtiene un camino mínimo en la rejilla, no necesariamente el camino continuo más corto.

Después se quitan puntos intermedios de la misma recta y se conservan las esquinas, sin crear diagonales. El controlador gira hacia cada esquina, consulta el contacto antes de cada avance y comprueba la posición para detenerse al llegar. Si un tramo falla, no sigue con el resto de la ruta.

#### Qué muestran las pruebas

La tabla 2 recoge un recorrido por escenario. Poses cuenta las posiciones de A\*, incluyendo el origen. Tramos cuenta las rectas después de quitar puntos intermedios. Longitud y error se expresan en unidades del mundo; el error compara la posición final con el destino ejecutado en la rejilla. Colisiones es el contador del simulador.

| Mapa  | Origen → destino          | Poses | Tramos | Longitud | Error | Colisiones |
|-------|---------------------------|-------|--------|----------|-------|------------|
| cfg_0 | (23.0, 7.0) → (2.0, 26.0) | 81    | 3      | 40.0     | 0.0   | 0          |
| cfg_1 | (2.0, 2.0) → (23.0, 23.0) | 85    | 4      | 42.0     | 0.0   | 0          |
| cfg_2 | (2.0, 2.0) → (39.0, 40.0) | 151   | 4      | 75.0     | 0.0   | 0          |
| cfg_3 | (2.0, 2.0) → (12.0, 12.0) | 41    | 4      | 20.0     | 0.0   | 0          |

Tabla 2. Navegación sobre mapas descubiertos en A. Los pares mostrados están alineados con la rejilla: en estos casos no hay ajuste de los extremos.

Por ejemplo, en cfg\_2 hay 151 poses: son 150 avances de 0,5 unidades, es decir, 75 unidades de recorrido. Tras comprimirlo quedan 4 tramos rectos. En los cuatro casos de la tabla 2 el error final y las colisiones registradas son cero.

## 4. Apartado C: conducir y repetir

### Guardar puntos de paso

Un waypoint es un punto de paso que queremos que el robot visite después. No se guarda cada posición de la conducción: se guarda el inicio, los puntos marcados con W y el punto final al salir. El JSON incluye nombre de ruta, escenario, fecha y lista ordenada de coordenadas. Dos puntos consecutivos iguales no se repiten.

La tabla 3 resume el teclado. Los giros de 90 grados mantienen las cuatro direcciones principales cuando se parte de orientación cero. Soltar la flecha de avance detiene el robot; S guarda sin cerrar la grabación.

| Tecla               | Acción                                             |
|---------------------|----------------------------------------------------|
| Flecha arriba       | Avanzar mientras se mantiene pulsada.              |
| Izquierda / derecha | Girar +90 / -90 grados por pulsación.              |
| W                   | Guardar la posición actual como waypoint.          |
| S                   | Guardar la ruta y seguir conduciendo.              |
| Q / cerrar ventana  | Guardar el punto final y salir.                    |
| D                   | Mostrar u ocultar los rectángulos de los sensores. |

Tabla 3. Controles del modo record de apartado\_c.py.

Cada punto aparece con una X azul y su número, empezando por el inicio como punto 1. En replay el siguiente objetivo es rojo. Si la ruta vuelve a una coordenada, se muestran allí todos sus números. La numeración corresponde al archivo, no a las esquinas internas del camino de A\*.

### Reproducir la ruta

replay carga mapa y ruta, comprueba que correspondan al mismo escenario y usa el A\* y el controlador compartidos entre puntos consecutivos. No repite pulsaciones ni garantiza seguir exactamente la trayectoria manual: puede rodear un mueble por otro camino.

C tiene una diferencia importante respecto a B: aproxima cada waypoint a una pose libre conectada con el robot. Puede acabar en un punto distinto al guardado si el mapa no lo contiene. `max_snap_distance` informa del mayor ajuste; una línea y un círculo rojos muestran el objetivo ejecutado cuando cambia. El error final compara la llegada con ese objetivo ajustado.

La ruta de prueba de `cfg_3` tiene 5 waypoints y 4 trayectos entre ellos. El mayor ajuste fue 0.0 unidades y el error final fue 0.0 unidades, con 0 colisiones. Se ejecutaron 100 ciclos de avance del controlador; no es una duración en segundos.

## 5. Pruebas y resultados

### Cómo se mide la cobertura

Al terminar A, una función de evaluación consulta la geometría real y calcula qué poses libres están conectadas con el inicio. Después calcula las celdas bajo las muestras de sus huellas. Es la referencia con la que se compara lo visitado. Se calcula después de explorar: no ayuda a elegir movimientos ni a crear la matriz.

La tabla 4 distingue ambas medidas. Poses alcanzables cuenta posiciones diferentes del centro; suelo alcanzable cuenta celdas diferentes que se pueden cubrir. Cobertura es el porcentaje de ese suelo cubierto. Omitidas cuenta celdas alcanzables no cubiertas. Pasos cuenta avances físicos, incluidos retrocesos, por lo que puede superar el número de poses.

| Mapa  | Poses alcanzables | Suelo alcanzable | Cobertura | Omitidas | Pasos | Colisiones |
|-------|-------------------|------------------|-----------|----------|-------|------------|
| cfg_0 | 1308              | 416              | 100.0%    | 0        | 2614  | 0          |
| cfg_1 | 1491              | 450              | 100.0%    | 0        | 2980  | 0          |
| cfg_2 | 4417              | 1306             | 100.0%    | 0        | 8832  | 0          |
| cfg_3 | 273               | 117              | 100.0%    | 0        | 544   | 0          |

Tabla 4. Cobertura de A en los cuatro escenarios originales, con el inicio definido para cada uno. Suelo alcanzable y omitidas se expresan en celdas.

En cfg\_0 hay 1.308 posiciones distintas del centro, pero 416 celdas de suelo alcanzables. Se hacen 2.614 avances porque el robot vuelve por las ramas exploradas. Como muestra la tabla 4, no quedan celdas alcanzables omitidas. La comparación de poses también da 100 % y no encuentra visitas fuera de la referencia en estos cuatro casos.

### Por qué puede haber zonas grises

El 100 % no se refiere a todas las celdas del piso. Una habitación aislada o un hueco demasiado estrecho no pertenece al conjunto alcanzable desde el inicio. El gris de la figura 1 indica que no se ha registrado cobertura; por sí solo no permite decidir si hay un obstáculo o suelo libre al que no se puede llegar.

### Pruebas automáticas y diagnóstico

Los tests comprueban que dos robots no compartan sensores, que guardar y cargar conserve los datos, que la matriz se cree al final y que A\* rodee obstáculos. También prueban rechazos de B y una ejecución completa de A, B y C en cfg\_3. Usan aserciones: comparaciones entre el resultado obtenido y el esperado que hacen fallar la prueba si no coinciden.

validar\_practica.py ejecuta los cuatro casos de las tablas 2 y 4 y el replay de ejemplo. Guarda medidas en resultados/diagnostico.json, pero no sustituye a los tests: registrar un resultado no equivale a comprobar automáticamente que sea correcto.

## 6. Retos, simplificaciones y mejoras

### En el apartado A

El reto principal fue separar lo que puede ocupar el centro de lo que cubre el cuerpo. Explorar solo desde coordenadas enteras dejaba salidas sin examinar en posiciones intermedias. La rejilla de medio paso corrige ese problema para este simulador. Aun así, la cobertura es la definida por sus muestras, no una medición continua de todo el suelo.

DFS termina si el espacio alcanzable es finito, los objetos no cambian y el inicio es válido y está alineado con la rejilla. No supone conocer las dimensiones, pero depende de que los contactos cierren el entorno. En un mundo abierto podría seguir descubriendo puntos; max\_probes limita los intentos para diagnóstico, no garantiza cobertura.

El recorrido repite pasos al volver atrás. Una mejora sería elegir mejor el orden de las zonas pendientes o reducir los retornos por caminos conocidos. Eso buscaría ahorrar distancia, no aumentar una cobertura que ya es completa en los casos medidos. La versión actual no hace una fase separada de seguimiento de paredes.

### En el apartado B

La dificultad fue no aceptar un obstáculo como destino por haber encontrado cerca un punto libre. Ahora se rechazan puntos bloqueados, desconocidos o desconectados. Queda la simplificación de ajustar localmente coordenadas entre muestras. El error cero significa llegada al objetivo de rejilla; no prueba llegada exacta a todos los puntos continuos posibles.

Los caminos solo son horizontales y verticales. Las diagonales podrían acortarlos, pero habría que comprobar todo el espacio que ocupa el cuerpo durante ese movimiento, no solo sus extremos. Con el mapa observado actual no se da esa comprobación por hecha.

### En el apartado C

Guardar puntos en vez de órdenes permite repetir la ruta con planificación, pero no conserva el camino manual exacto. Queda por decidir si C debería rechazar también cualquier waypoint no accesible, como B, en lugar de aproximarlos. Mostrar y medir el ajuste evita confundir lo solicitado con lo ejecutado, pero no elimina esa diferencia.

### Fuera del simulador

La posición es perfecta y los objetos son estáticos. En un robot real habría que corregir errores de posición y revisar el mapa ante obstáculos nuevos. Un sensor de distancia permitiría descubrir más sin acercarse a cada mueble. Hay batería en el simulador, pero no se ha implementado retorno autónomo al cargador.

## 7. Conclusiones

La principal lección es que un mapa para navegar no basta con que se parezca al dibujo del piso. Debe representar dónde cabe el robot y distinguir lo observado de lo desconocido. También hay que comprobar el suelo cubierto por su cuerpo por separado del recorrido de su centro.

A crea el mapa sin reservar una matriz del tamaño del escenario. B encuentra caminos en la rejilla conocida y comunica cuándo no puede llegar. C guarda puntos ordenados y usa el mismo planificador para reproducirlos. Las pruebas permiten comprobar estos comportamientos; sus resultados no se extienden automáticamente a un robot real o a cualquier escenario nuevo.

También hemos aprendido que llegar sin error a un objetivo no basta para evaluar una ruta: hay que comprobar si ese objetivo coincide con el solicitado y cuánto se ha ajustado. Por eso se distinguen las coordenadas originales de las ejecutadas, y se conservan por separado las medidas de cobertura, distancia y colisiones.

### Reproducción de resultados

Desde rds2026, con el entorno de Python activado y Pygame instalado:

```
python -m unittest discover -s tests -v  
python validar_practica.py
```

La primera orden ejecuta las pruebas automáticas de representación del mapa, planificación, rechazo de destinos y movimiento en el simulador. La segunda explora los cuatro escenarios originales, ejecuta los recorridos de B y reproduce la ruta de ejemplo de C. Guarda las medidas en resultados/diagnostico.json, que es el archivo utilizado para elaborar las tablas de esta memoria.

El diagnóstico se ejecuta sin ventana y sin limitar los ciclos por segundo para reducir el tiempo de cálculo. Se mantienen el paso físico de 0,5 unidades y las mismas comprobaciones de contacto de las ejecuciones con ventana. No se interpreta el número de ciclos como una duración real del recorrido.